

Joc de la Calculadora amb Programació Dinàmica

Pràctica 5

Serafí Nebot Ginard

Universitat de les Illes Balears (UIB)

Grau en Enginyeria Informàtica

Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu una aplicació Java que resol el joc de la calculadora mitjançant programació dinàmica. El joc parteix d'un total acumulat, una tecla jugada anteriorment i un jugador a qui toca moure; a cada torn només es poden prémer tecles de la mateixa fila o columna que la tecla anterior, i perd el jugador que arriba o supera el límit. S'han implementat dues variants de la mateixa recurrència: una versió *top-down* amb memoització i una versió *bottom-up* que precomputa tota la taula d'estats. L'aplicació separa model, vista i controlador, ofereix una interfície Swing amb recàlcul automàtic, teclats estàndard o aleatoris i una reproducció pas a pas de la partida òptima. També s'ha incorporat una eina de benchmark que mesura temps de CPU, temps real, estats calculats i variació aproximada de memòria. Els resultats confirmen que la versió *bottom-up* té un cost més predictable i proporcional a tota la taula, mentre que la versió *top-down* visita menys estats quan la partida inicial només arriba a una part de l'espai possible.

Termes d'índex—programació dinàmica, jocs combinatoris, memoització, taula *bottom-up*, Model-Vista-Controlador, Java Swing, benchmark

1. Introducció

La pràctica tracta un joc determinista sobre una calculadora. Hi ha un valor acumulat, un límit de pèrdua, un teclat rectangular i un conjunt de jugadors. En cada torn, el jugador prem una tecla per sumar el seu valor al total. Si el nou total arriba o supera el límit, aquest jugador perd immediatament. La restricció que fa interessant el problema és que, després de la primera jugada, només es poden emprar tecles situades a la mateixa fila o columna que la tecla jugada anteriorment.

L'objectiu principal és determinar quin jugador perd si tots juguen de manera òptima per evitar ser el perdedor. En partides de dos jugadors això també identifica el guanyador; en partides amb més jugadors només es reporta el perdedor, perquè hi pot haver més d'un supervivent i no hi ha un únic guanyador natural. La noció d'òptim emprada és local a cada jugador: en el seu torn, cada jugador tria un moviment que evita la seva pròpia derrota si existeix. No es modela cooperació entre jugadors ni preferències sobre quin altre jugador hauria de perdre.

La solució implementada no explora l'arbre de joc de manera ingènua. Com que el total acumulat sempre augmenta, el graf d'estats és acíclic i es pot resoldre amb programació dinàmica [1]. La pràctica compara dues formes de la mateixa idea: una a demanda amb memoització i una iterativa que omple tota la taula. A més, s'ha construït una aplicació d'escriptori completa amb patró Model-Vista-Controlador i una eina separada per obtenir dades experimentals.

2. Descripció del Problema

El teclat té amplada W i alçada H , amb dimensions entre 2 i 5. El nombre de tecles és $K = W \cdot H$ i cada tecla conté exactament un valor de l'interval $1, \dots, K$. La distribució estàndard imita una calculadora: per exemple, un teclat 3×3 mostra 7, 8, 9 a la fila superior, 4, 5, 6 al centre i 1, 2, 3 a la part inferior.

El valor especial 0 indica que encara no s'ha premut cap tecla. En aquest cas, totes les tecles són moviments vàlids. Si la darrera tecla jugada és $l > 0$, el conjunt de moviments vàlids $M(l)$ conté les tecles de la mateixa fila i de la mateixa columna que l , excloent l mateix. Per exemple, en el teclat 3×3 estàndard, si la darrera tecla és 6, els moviments vàlids són 3, 4, 5, 9.

La configuració d'una partida queda definida per:

- el teclat concret;
- el total inicial t_0 ;
- el límit de pèrdua L ;
- la darrera tecla jugada l_0 , o 0 si no n'hi ha;
- el nombre de jugadors P ;
- el jugador que ha de moure.

La implementació permet teclats estàndard i teclats aleatoris. En mode aleatori, la disposició continua essent una permutació vàlida de $1, \dots, K$; només canvia la posició de cada valor. Això obliga la generació aleatòria a preservar dues invariants: no hi pot haver valors repetits i no pot faltar cap tecla de l'interval. A la interfície, el botó *Nou aleatori* canvia la llavor usada per generar la permutació, mentre que

el benchmark empra teclats estàndard per fer comparables les mesures.

3. Fonaments Teòrics

3.1. Estat i Recurrència

L'estat resolt pels algorismes és:

$$S = (t, l, j) \quad (1)$$

on t és el total actual, l és la darrera tecla jugada i j és el jugador a qui toca moure. La funció dinàmica $F(t, l, j)$ retorna l'índex del jugador que perdrà des d'aquest estat si tots juguen òptimament.

Per a cada moviment vàlid $m \in M(l)$ es defineix el successor segur:

$$\text{succ}(t, l, j, m) = (t + m, m, (j + 1) \bmod P) \quad (2)$$

sempre que $t + m < L$. Si $t + m \geq L$, el moviment és terminal i fa perdre directament el jugador j , de manera que no és un moviment útil per evitar la pròpia derrota. Per abreujar la notació, escrivim $j' = (j + 1) \bmod P$.

El conjunt de moviments que salven el jugador actual és:

$$A(t, l, j) = \{m \in M(l) \mid t + m < L \wedge F(t + m, m, j') \neq j\} \quad (3)$$

La recurrència implementada és:

$$F(t, l, j) = \begin{cases} j, & \text{si } A(t, l, j) = \emptyset \\ F(t + m^*, m^*, j'), & \text{si } A(t, l, j) \neq \emptyset \end{cases} \quad (4)$$

on m^* és el primer moviment de $A(t, l, j)$ en ordre ascendent. Aquest desempat no canvia si el jugador pot evitar perdre, però fa que la reproducció de la partida sigui determinista i que els dos algorismes mostrin exactament la mateixa línia òptima.

3.2. Per què Programació Dinàmica

Una exploració directa podria recalculer moltes vegades el mateix estat, perquè seqüències diferents de tecles poden conduir al mateix triple (t, l, j) . La programació dinàmica evita aquesta repetició guardant el valor de cada estat una sola vegada.

El detall clau és que no hi ha cicles: qualsevol moviment vàlid incrementa el total en un valor positiu. Per tant, tots els successors segurs d'un estat tenen un total més gran. Això permet resoldre el problema tant de forma recursiva amb memoització (*top-down*) com de forma iterativa baixant de $L - 1$ fins a 0 (*bottom-up*).

En la variant *top-down*, el programa parteix només de l'estat inicial i aplica la recurrència de manera recursiva.

Quan apareix un estat nou, es calcula el seu perdedor i es guarda en una memòria; si el mateix estat torna a aparèixer per un altre camí de joc, es reutilitza el valor ja calculat. Per tant, aquesta variant calcula només la part de l'espai d'estats que és necessària per respondre la consulta inicial.

En la variant *bottom-up*, en canvi, no es comença per l'estat inicial, sinó que s'omple tota la taula d'estats possibles. Es processen primer els totals més propers al límit, perquè en aquests casos els successors ja són terminals o molt simples. Després es baixa progressivament fins a 0. Quan es calcula un estat (t, l, j) , tots els successors amb total $t + m$ ja estan resolts, i per això el valor es pot obtenir consultant directament la taula.

3.3. Correcció dels Algorismes

La correcció es basa en inducció sobre la distància fins al límit. Els estats amb totals propers a L són els més simples: si tots els moviments vàlids arriben o superen el límit, qualsevol jugada fa perdre el jugador actual i, per tant, $F(t, l, j) = j$. Aquest és el cas base implícit de la recurrència.

Per al pas inductiu, es considera un estat (t, l, j) amb successors segurs. Com que tots els moviments sumen un valor positiu, qualsevol successor té total $t + m > t$ i està més a prop del límit. Per hipòtesi inductiva, el valor $F(t + m, m, j')$ d'aquests successors ja descriu correctament qui perd des d'allà. Si existeix qualche moviment segur amb $F(t + m, m, j') \neq j$, el jugador actual pot evitar ser el perdedor triant-lo. Si no existeix cap moviment d'aquest tipus, tots els camins possibles acaben fent perdre el jugador actual, i per això la resposta correcta és j .

Les dues variants implementen aquesta mateixa recurrència. La versió *top-down* la avalua sota demanda i memoïtza els resultats; la versió *bottom-up* calcula els mateixos valors en un ordre que garanteix que cada successor segur ja està resolt. Per això, si parteixen de la mateixa configuració, han de retornar el mateix perdedor i la mateixa primera jugada segons el criteri de desempat.

3.4. Complexitat

El nombre màxim d'estats no terminals és:

$$S_{max} = L \cdot (K + 1) \cdot P \quad (5)$$

El factor $K + 1$ apareix perquè l pot ser qualsevol tecla o el valor especial 0. Per estat, el nombre de moviments vàlids és com a màxim:

$$B = \max(K, W + H - 2) \quad (6)$$

El cas K correspon al primer moviment, quan $l = 0$; la resta d'estats tenen com a màxim $W + H - 2$ moviments perquè només es pot triar fila o columna.

La versió *top-down* només expandeix els estats que la recurrència arriba a necessitar des de la configuració inicial. Si R és aquest nombre d'estats visitats, el cost és:

$$T_{TD} = O(R \cdot B), \quad M_{TD} = O(R) \quad (7)$$

Cal distingir R de S_{max} , que és la mida de l'espai complet d'estats que es podria construir, mentre que R és només el nombre d'estats que la recursió arriba a visitar i guardar per a una configuració inicial concreta. Per tant, sempre es compleix $R \leq S_{max}$. En el pitjor cas, si l'exploració des de l'estat inicial acaba necessitant pràcticament tota la taula, R pot arribar a ser igual o molt proper a S_{max} ; però en molts casos és menor, i aquest és el principal avantatge pràctic de la variant *top-down*.

La versió *bottom-up* omple tota la taula independentment de si tots els estats seran consultats des de la partida inicial:

$$T_{BU} = O(S_{max} \cdot B), \quad M_{BU} = O(S_{max}) \quad (8)$$

Encara que la implementació *bottom-up* guarda dues taules tridimensionals, una per al jugador perdedor i una altra per al moviment triat, cadascuna té mida proporcional a S_{max} . Per tant, la memòria total és $O(2 \cdot S_{max})$, que asimptòticament continua sent $O(S_{max})$.

4. Disseny i Implementació

4.1. Arquitectura MVC

L'aplicació segueix el patró Model-Vista-Controlador [2]. El punt d'entrada `Main` crea la finestra `Swing` i després crea el controlador. La finestra no coneix cap classe concreta de controlador; només emet esdeveniments a través de la interfície `GameViewListener`. Això manté la vista separada de les regles del joc.

La decisió de fer aquesta separació no és només organitzativa. El projecte té dues parts molt diferents: d'una banda hi ha la solució del problema amb algorismes de programació dinàmica; de l'altra, una interfície interactiva que ha de respondre a cada canvi de l'usuari. Si la vista contingués lògica de joc, seria difícil validar els casos límit sense obrir finestres. Si el model conegués `Swing`, els algorismes no es podrien reutilitzar en el benchmark. Per això la dependència sempre va en la mateixa direcció: la vista exposa una abstracció, el controlador la consumeix i el model no coneix cap classe gràfica.

La responsabilitat de cada capa és:

- **Model:** conté les dades immutables del problema (`GameConfig`, `GameState`, `Keypad`) i els algorismes de programació dinàmica.
- **Vista:** implementada per `GameFrame`; dibuixa el teclat, els controls, els resultats i la reproducció pas a pas.
- **Controlador:** `GameController` llegeix el formulari, valida l'entrada, crea el teclat i l'algorisme seleccionat, i retorna un `GameResult` llest per mostrar.

La vista recalcula automàticament quan canvia qualsevol paràmetre, de manera que no cal un botó de càlcul. Si la

configuració és invàlida, el controlador captura l'error de validació i envia un missatge a la vista en lloc d'executar cap algorisme.

El flux complet d'una modificació de la GUI és el següent:

- 1) L'usuari canvia un camp, un selector o prem *Nou aleatori*.
- 2) `GameFrame` notifica `configurationChanged()` al seu `GameViewListener`.
- 3) `GameController` llegeix un `GameInput` de la vista.
- 4) El controlador construeix el `Keypad`, valida la configuració i crea un `GameConfig` amb índexs interns de jugador basats en 0.
- 5) Segons el selector d'algorisme, crea `TopDownDPSolver` o `BottomUpDPSolver`.
- 6) L'algorisme calcula el perdedor, la línia de reproducció i el nombre d'estats calculats.
- 7) El controlador encapsula les dades en un `GameResult` i la vista redibuixa el teclat, els resultats i el pas actual de la reproducció.

La Taula 1 resumeix tots els tipus definits a `src/main/java`, no només el nucli algorítmic. Això deixa explícit com es reparteix la responsabilitat entre el punt d'entrada, el model, el controlador, la vista i el codi d'anàlisi experimental.

4.2. Entrada, Validació i Conversió

La vista treballa amb `GameInput`, un tipus *record* de Java pensat per representar el formulari tal com el veu l'usuari. En aquest objecte, el jugador inicial usa indexació que comença a 1, perquè a la interfície és natural mostrar "Jugador 1", "Jugador 2", etc. En canvi, els algorismes treballen amb indexació que comença a 0, ja que això simplifica el càlcul del següent torn amb l'operació mòdul. Aquesta conversió es fa només al controlador, en el moment de crear `GameConfig`.

La validació està repartida en dos nivells. El controlador comprova les regles que depenen de la interacció de diversos camps: que el límit sigui superior al total inicial, que la darrera tecla sigui 0 o pertanyi al teclat, i que el jugador inicial estigui dins el rang de jugadors. Després, `GameConfig` repeteix les invariants essencials del model. Aquesta duplicació parcial és intencionada: el controlador dona missatges d'error pensats per l'usuari, mentre que el model es protegeix de qualsevol ús incorrecte des del benchmark o altres crides internes.

També hi ha una diferència entre `GameInput` i `GameResult`. `GameInput` és entrada bruta de la vista; `GameResult` és sortida preparada per mostrar. Inclou el teclat materialitzat, els moviments vàlids de l'estat inicial, el jugador perdedor, el guanyador si només hi ha dos jugadors, el nombre d'estats calculats i la llista de passos de reproducció. La vista no ha de deduir cap regla del joc a partir d'aquest resultat; simplement el presenta.

Taula 1. TOTS ELS TIPUS PRINCIPALS DEFINITS AL PROJECTE.

Paquet	Tipus	Nom	Responsabilitat
arrel	classe	Main	Punt d'entrada de la GUI; crea la vista i el controlador.
arrel	classe	BenchmarkMain	Punt d'entrada de consola per executar el benchmark i imprimir CSV.
model	classe	Keypad	Teclat immutable, disposició de valors i moviments vàlids precomputats.
model	enumeració	KeypadMode	Distingeix entre teclat estàndard i teclat aleatori.
model	enumeració	SolverMode	Selecciona l'algorisme <i>top-down</i> o <i>bottom-up</i> .
model	record	GameInput	Configuració llegida de la vista abans de validar-la.
model	record	GameConfig	Configuració validada i preparada per ser usada pels algorismes.
model	record	GameState	Estat del joc (t, l, j) emprat com a clau conceptual de la DP.
model	record	GameResult	Resultat preparat per a la vista: perdedor, moviments inicials, estatística i reproducció.
model	record	ReplayStep	Informació d'un pas concret de la reproducció òptima.
model	classe abstracta	Solver	Contracte comú dels algorismes i lògica compartida de reproducció.
model	classe	TopDownDPSolver	Implementació recursiva amb memoització a demanda.
model	classe	BottomUpDPSolver	Implementació iterativa que omple tota la taula d'estats.
controller	classe	GameController	Llegeix la configuració de la vista, valida, crea l'algorisme i retorna un <code>GameResult</code> .
view	interfície	GameView	Contracte que la vista exposa al controlador.
view	interfície	GameViewListener	Contracte d'esdeveniments que la vista emet cap al controlador.
view	classe	GameFrame	Finestra Swing amb formulari, teclat, resultats i navegació de la reproducció.
analysis	classe	SolverBenchmark	Executa sèries de benchmark i agrega mesures per configuració.
analysis	record	SolverMeasurement	Una fila agregada de benchmark amb temps, memòria i estats calculats.

4.3. Representació dels Estats

La classe `GameState` és un tipus *record* immutable amb tres camps: total actual, darrera tecla i torn. Aquesta immutabilitat és important per a la versió *top-down*, perquè els estats s'usen com a claus dins un `HashMap`. Si un estat fos mutable, canviar-ne un camp després d'inserir-lo al mapa podria trencar la correspondència entre `hashCode`, `equals` i la posició interna de la taula hash.

La implementació evita guardar estats terminals amb $t \geq L$. Quan un moviment arriba o supera el límit, el perdedor és conegut immediatament: és el jugador que acaba de moure. Això redueix la taula a totals $0, \dots, L - 1$ i simplifica el *bottom-up*, perquè totes les posicions guardades representen estats on encara s'ha de jugar.

El camp `lastPlayed` pot valer 0. Aquest valor no és una tecla real, sinó un sentinella que representa el primer moviment. Per això la taula *bottom-up* usa la dimensió `keyCount + 1`: les posicions de 1 a K corresponen a tecles, i la posició 0 correspon a l'estat inicial sense tecla prèvia.

4.4. Model del Teclat

`Keypad` és immutable. El constructor valida que les dimensions estiguin dins el rang permès, que hi hagi exactament $W \cdot H$ valors i que aquests formin una permutació sense duplicats. Durant la construcció es precomputen els moviments vàlids de cada tecla. Això fa que els algorismes puguin consultar `moves(lastPlayed)` sense tornar a recórrer la graella.

El teclat estàndard es construeix per files de dalt a baix amb els valors més alts a la part superior. El teclat aleatori utilitza una permutació de les mateixes tecles; a la GUI, el botó *Nou aleatori* canvia la llavor i força una nova disposició.

Internament la disposició es guarda en un vector unidimensional en ordre per files. La posició (r, c) es transforma en l'índex $r \cdot W + c$. Aquesta representació és compacta

i evita crear objectes per a cada casella. Per calcular els moviments d'una tecla, el constructor localitza la seva fila i columna i copia tots els valors de la mateixa fila i de la mateixa columna, excepte la pròpia tecla. El resultat es guarda ordenat. L'ordre ascendent és rellevant perquè els algorismes usen el primer moviment segur com a criteri de desempat i, a més, permet aturar el recorregut quan apareix el primer moviment terminal: si un valor ja fa que $t + m \geq L$, tots els moviments següents també seran terminals.

El cost de precomputar moviments és baix. Hi ha K tecles i per a cada una es recorren $W - 1$ posicions de fila i $H - 1$ posicions de columna. Per tant, el cost de preparació del teclat és $O(K(W + H))$, molt menor que el cost potencial de resoldre tots els estats quan L o P creixen. A canvi, cada consulta `moves(lastPlayed)` pot retornar una còpia defensiva ja preparada. Les còpies defensives impedeixen que una classe externa modifiqui el vector intern i alteri les regles del joc després de construir el teclat.

4.5. Algorismes

Els dos algorismes hereten de `Solver`, que defineix el contracte comú: calcular el perdedor, consultar el perdedor d'un estat arbitrari, indicar quants estats s'han calculat i construir una reproducció òptima. El mètode de reproducció també és comú, de manera que les dues implementacions comparteixen la mateixa política de desempat.

`TopDownDPSolver` implementa la recurrència de l'Eq. 4 amb un mapa `HashMap`. Quan rep una consulta, primer mira si l'estat ja és a la memòria; si no hi és, avalua els moviments vàlids i guarda el perdedor calculat. Aquesta variant és especialment útil quan l'estat inicial no necessita visitar tota la taula teòrica.

El mètode central de la versió *top-down* és `loser(state)`. La seva estructura reflecteix directament la recurrència: per defecte assumeix que perd el jugador actual; després revisa els moviments vàlids en ordre ascendent. Els moviments terminals no es desenvolupen

perquè ja fan perdre el jugador actual, i com que els moviments estan ordenats, en trobar el primer moviment terminal es pot aturar tot el bucle. Per a cada moviment segur es consulta recursivament el successor. Si el perdedor del successor no és el jugador actual, s'ha trobat una jugada que evita la derrota pròpia i es pot aturar el bucle. Si cap moviment compleix aquesta condició, l'assumpció inicial queda confirmada.

Algorithm 1 Esquema de l'algorisme top-down.

```

1: function LOSER( $t, l, j$ )
2: if estat és a memo then
3:   return valor memoitzat
4: end if
5:  $perdedor \leftarrow j$ 
6: for all  $m \in M(l)$  en ordre ascendent do
7:   if  $t + m \geq L$  then
8:     break
9:   end if
10:   $p \leftarrow \text{LOSER}(t + m, m, (j + 1) \bmod P)$ 
11:  if  $p \neq j$  then
12:     $perdedor \leftarrow p$ ; break
13:  end if
14: end for
15: guarda  $perdedor$  a memo
16: return  $perdedor$ 

```

La recursió no pot entrar en cicles perquè el total augmenta estrictament. La profunditat màxima està limitada pel nombre de jugades abans d'arribar a L ; en el pitjor cas, si sempre es prem la tecla 1, és com a màxim $L - t_0$. En les execucions realitzades això no ha creat problemes de pila; a més, la memoització evita que branques diferents recalculin els mateixos successors.

`BottomUpDPSolver` crea dues taules tridimensionals de enters: una per al perdedor de cada estat i una per al moviment triat. Les dimensions són $L \cdot (K + 1) \cdot P$. El bucle principal recorre els totals de `límit - 1` fins a 0; com que qualsevol successor segur té total més gran, el valor ja està disponible a la taula.

La taula `losers` guarda el jugador perdedor de cada estat. La taula `moves` guarda el moviment escollit per a la reproducció. Aquesta segona taula no és imprescindible per respondre qui perd, però evita tornar a recórrer els moviments quan la GUI demana la línia òptima. El cost extra és una altra taula d'enters amb les mateixes dimensions, acceptable dins els límits de teclat i límit de la pràctica.

Algorithm 2 Esquema de l'algorisme bottom-up.

```

1: for  $t = L - 1$  downto 0 do
2:   for  $l = 0$  to  $K$  do
3:     for  $j = 0$  to  $P - 1$  do
4:        $perdedor \leftarrow j$ 
5:        $moviment \leftarrow$  primer element de  $M(l)$ 
6:       for all  $m \in M(l)$  en ordre ascendent do
7:         if  $t + m \geq L$  then
8:           break
9:         end if
10:        if  $losers[t + m][m][(j + 1) \bmod P] \neq j$  then
11:           $perdedor \leftarrow losers[t + m][m][(j + 1) \bmod P]$ 
12:           $moviment \leftarrow m$ ; break
13:        end if
14:      end for
15:       $losers[t][l][j] \leftarrow perdedor$ 
16:       $moves[t][l][j] \leftarrow moviment$ 
17:    end for
18:  end for
19: end for

```

La part crítica és l'ordre del primer bucle. Si es recorreguessin els totals de 0 cap amunt, un estat podria necessitar un successor que encara no està calculat. Recorrent de $L - 1$ cap a 0, qualsevol successor segur $t + m$ és més gran que t i ja ha estat emplenat. Els successors terminals no necessiten entrada a la taula, perquè la seva resposta és directa. Igualment, el recorregut dels moviments es pot aturar al primer successor terminal perquè els valors legals es processen en ordre ascendent.

4.6. Reproducció de la Partida Òptima

La GUI no només ha de mostrar el perdedor, sinó també una possible partida òptima. Aquesta funcionalitat es concentra a `Solver.replay()`, compartida pels dos algorismes. El mètode comença amb l'estat inicial i afegeix un `ReplayStep` sense moviment, que representa la situació de partida. Després repeteix el procés següent: demanar `chosenMove(state)`, sumar el moviment, crear un nou pas de reproducció i, si el total encara és inferior al límit, avançar al següent torn.

Cada `ReplayStep` conté les dades que necessita la vista: número de pas, total anterior, total nou, darrera tecla, jugador que ha mogut, moviment fet, següent jugador, moviments vàlids del pas següent i indicador de terminalitat. Això evita que `GameFrame` hagi de reconstruir informació del model. Per exemple, quan un pas és terminal, la llista de moviments vàlids següents és buida i la vista només ha d'indicar que la partida ha acabat.

El desempat també és important en aquesta part. Si hi ha diversos moviments que eviten perdre, es tria el primer en ordre ascendent. Si no n'hi ha cap, tots els moviments són perdedors per al jugador actual; en aquest cas `chosenMove` retorna el primer moviment vàlid com a línia representativa. Això dona una partida reproduïble i fa que es pugui comparar la reproducció sencera entre *top-down* i *bottom-up*.

4.7. Interfície Gràfica

La interfície Swing [3] està organitzada en panells de configuració, teclat, resultats i reproducció. L'usuari pot modificar dimensions, total inicial, límit, darrera tecla, nombre de jugadors, jugador inicial, mode de teclat i mode d'algorisme. El teclat es mostra com una calculadora real: les tecles vàlides de l'estat actual es ressalten en blau, la darrera tecla jugada en groc i les tecles invàlides en gris.

El panell de resultats mostra el jugador que perd, el guanyador quan $P = 2$, els moviments vàlids inicials i el nombre d'estats calculats. La reproducció permet avançar i retrocedir pels passos de la partida òptima. A cada pas es veu qui ha mogut, quina tecla ha premut, quin total s'ha obtingut i quin jugador mou a continuació. Quan el pas és terminal, la vista indica que aquell moviment ha arribat o superat el límit.

La interfície està pensada perquè el resultat sigui visible sense passos manuals innecessaris. Els camps numèrics utilitzen components de selecció amb límits raonables, els desplegable controlen el mode de teclat i el mode d'algorisme, i qualsevol canvi llança una nova resolució. Això permet comparar ràpidament la mateixa partida amb *top-down* i *bottom-up*, o veure com canvia el perdedor en modificar el límit.

El dibuix del teclat usa el resultat del controlador, no recalcula moviments. La vista rep el vector de moviments vàlids i el pas de reproducció actual. Amb aquesta informació assigna colors: blau per a tecles vàlides, groc per a la darrera tecla del pas i gris per a tecles no disponibles. Aquesta codificació visual ajuda a comprovar manualment que la restricció de fila i columna s'està aplicant bé.

Per a partides amb més de dos jugadors, el camp *Guanya* mostra un guió. Aquesta decisió evita donar una resposta enganyosa. En un joc de $P > 2$, saber qui perd no determina un únic guanyador; simplement determina quin jugador queda eliminat per la seqüència òptima considerada. El camp *Perd* sí que es manté per a qualsevol P .

4.8. Gestió d'Errors i Casos Límit

Els errors d'entrada es gestionen sense interrompre l'aplicació. Si el límit no és superior al total inicial, si la darrera tecla no pertany al teclat o si el jugador inicial és invàlid, el controlador no crea cap algorisme i la vista mostra el missatge corresponent. Això és important perquè el recàlcul és automàtic: amb un formulari interactiu és normal que l'usuari passi temporalment per valors invàlids mentre està editant.

També s'han tractat dos casos especials del joc. El primer és el moviment inicial amb `lastPlayed = 0`; en aquest cas `Keypad.moves(0)` retorna totes les tecles ordenades. El segon és l'estat on tots els moviments vàlids són terminals o condueixen a perdre: l'algorisme ha de retornar el jugador actual com a perdedor, però encara ha de poder construir una reproducció. Per això es guarda un moviment de reserva, el primer vàlid, que només s'usa quan no hi ha cap moviment salvador.

4.9. Benchmark

Per separar les mesures de rendiment de la GUI s'ha creat el paquet `analysis`.

`SolverBenchmark` executa els dos algorismes sobre una llista de configuracions, aplica escalfaments previs i després calcula mitjanes de les repeticions mesurades. `BenchmarkMain` imprimeix els resultats en CSV, cosa que permet copiar-los directament a la memòria o a una eina externa.

La comanda utilitzada per a les mesures de la Secció 5 és:

```
1 mvn -q exec:java \
2   -Dexec.mainClass="com.serafinebot.p5.
3     BenchmarkMain" \
4     -Dexec.args="3 10"
```

Listing 1. Execució del benchmark de P5.

El benchmark mesura la materialització del teclat i de la configuració, la construcció de l'algorisme, el càlcul del perdedor i la generació de la reproducció. No mesura el dibuix de la interfície Swing. El temps de CPU s'obté amb `ThreadMXBean`; la memòria és una diferència aproximada de memòria dinàmica abans i després de cada execució, útil per observar tendències.

Les configuracions per defecte del benchmark combinen dues sèries. La primera recorre teclats quadrats de 2×2 fins a 5×5 amb $L = 31$; la segona fixa el teclat a 3×3 i fa créixer el límit fins a 10000. Totes parteixen de total 0, sense darrera tecla, dos jugadors i jugador inicial 1. S'ha triat aquest conjunt perquè creix en dues direccions diferents: augmentar el límit fa créixer la dimensió de totals, mentre que augmentar el teclat fa créixer tant el nombre de valors de `lastPlayed` com la ramificació del primer moviment.

Cada fila de CSV inclou el nombre d'estats de la taula completa i el nombre d'estats realment calculats per l'algorisme. En *bottom-up* aquests dos valors coincideixen sempre. En *top-down*, el segon valor és la mida del mapa de memoització després de resoldre i generar la reproducció. Aquesta diferència és la magnitud més important per interpretar els resultats, perquè és independent del soroll temporal propi d'executar microexecucions de rendiment sobre la JVM. A més, el benchmark no inclou el cost de pintar la GUI ni la interacció de l'usuari; només mesura la preparació mínima de l'entrada, la part algorísmica i la construcció de la reproducció.

5. Resultats i Discussió

5.1. Protocol Experimental

Les mesures s'han pres amb la comanda del Llistat 1. Per a cada configuració s'han executat 3 escalfaments i 10 repeticions mesurades. Els teclats són estàndard, el total inicial és 0, la darrera tecla és 0, el jugador inicial és el jugador 1 i hi ha 2 jugadors. S'han variat dues dimensions de l'entrada: la mida del teclat i el límit de perduda. Per estudiar el límit s'ha usat una sèrie llarga

$L \in \{31, 60, 100, 200, 500, 1000, 2000, 5000, 10000\}$ amb teclat 3×3 .

Com que les instàncies són petites, els temps absoluts estan a l'escala de microsegons i poden tenir soroll de JVM, planificador i recol·lector de brossa. Per això la discussió dona més pes al nombre d'estats calculats, que és una mesura directa de la feina estructural feta per cada algorisme. Els temps de CPU i la variació de memòria dinàmica es mantenen com a indicadors pràctics complementaris.

5.2. Creixement amb el Límit

La Taula 2 fixa el teclat a 3×3 i mostra què passa quan creix el límit L . La taula completa de l'algorisme *bottom-up* creix de forma exactament lineal amb L , tal com prediu l'Eq. 5. En canvi, la versió *top-down* visita menys estats, però també creix quan el límit permet partides més llargues.

Taula 2. SÈRIE DE LÍMITS AMB TECLAT 3×3 I 2 JUGADORS.

L	Taula	TD est.	TD/taula	TD CPU (μs)	BU CPU (μs)
31	620	314	50,6%	189,2	120,8
60	1.200	743	61,9%	89,4	64,9
100	2.000	1.408	70,4%	162,7	94,6
200	4.000	2.438	61,0%	270,0	265,0
500	10.000	5.438	54,4%	546,3	460,5
1.000	20.000	10.438	52,2%	605,8	766,0
2.000	40.000	20.438	51,1%	1.485,8	1.486,0
5.000	100.000	50.438	50,4%	5.246,8	4.015,3
10.000	200.000	100.438	50,2%	10.317,1	7.934,6

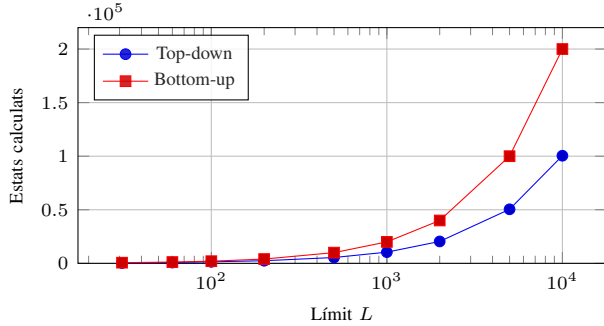


Figura 1. Creixement dels estats calculats amb teclat 3×3 .

El resultat més rellevant és que *bottom-up* calcula sempre el 100% dels estats de la taula. *Top-down* en calcula una fracció menor: en aquesta sèrie comença entre el 50% i el 70% i, per als límits grans, s'estabilitza prop del 50%. Això mostra que el benefici de la memoització a demanda no és només un efecte de límits petits. La Fig. 1 mostra que els dos algorismes creixen amb L , però *bottom-up* segueix exactament la taula completa, mentre que *top-down* queda sistemàticament per davall.

5.3. Creixement amb la Mida del Teclat

La Taula 3 fixa $L = 31$ i mostra el creixement quan augmenta el nombre de tecles. El nombre teòric d'estats

creix amb $K + 1$, i el factor de ramificació també creix perquè al primer moviment totes les tecles són vàlides.

Taula 3. ESTATS CALCULATS QUAN CREIX EL TECLAT, AMB $L = 31$.

Teclat	K	B	Taula	TD estats	TD/taula
2×2	4	4	310	62	20,0%
3×3	9	9	620	314	50,6%
4×4	16	16	1054	416	39,5%
5×5	25	25	1612	563	34,9%

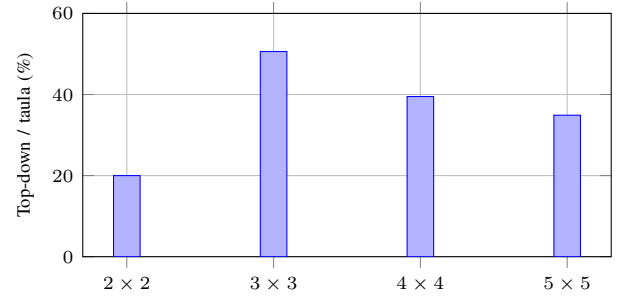


Figura 2. Percentatge de la taula completa visitat per *top-down* amb $L = 31$.

L'algorisme *bottom-up* coincideix sempre amb la columna *Taula*, perquè no filtra estats segons l'assolibilitat des de l'estat inicial. La versió *top-down*, en canvi, queda entre el 20% i el 50,6% en aquests casos. El percentatge no és monòton perquè depèn de les sumes concretes que es poden aconseguir amb cada teclat i de la política de desempat, com es veu a la Fig. 2. Tot i així, confirma que la memoització evita una part important de la taula completa.

5.4. Temps de CPU i Memòria

La Fig. 3 representa els temps de CPU de la mateixa sèrie llarga de límits de la Taula 2. Els temps absoluts continuen essent petits, però amb nous valors de L ja es veu una tendència creixent. També es veu que el temps no segueix exactament el nombre d'estats: les constants de la JVM, el cost del HashMap i la localitat dels vectors primitius fan que hi hagi canvis de lideratge entre les dues variants.

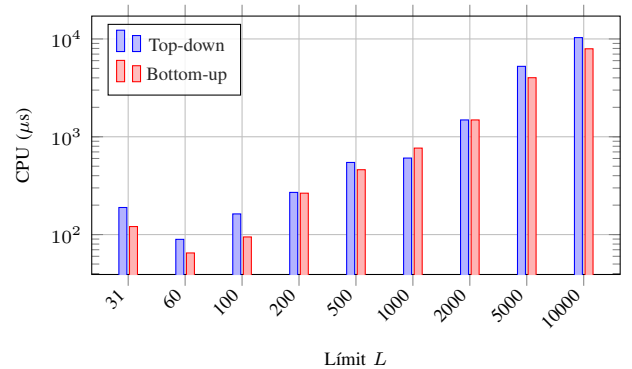


Figura 3. Temps de CPU amb teclat 3×3 per a la sèrie llarga de límits.

En memòria, la mesura directa de la memòria dinàmica del procés no és prou estable per representar-la com a gràfica: la JVM pot reutilitzar espai o activar el recol·lector de brossa entre repeticions. Per això la comparació principal de memòria és estructural. *Bottom-up* reserva dues taules de mida proporcional a S_{max} , mentre que *top-down* guarda R entrades dins un `HashMap`. Així, *top-down* sol tenir menys estats, però cada entrada té més sobrecàrrega; *bottom-up* guarda més posicions, però en vectors primitius més compactes.

5.5. Discussió General

Les dues variants són correctes i comparteixen la mateixa política de joc òptim. La diferència és quan i quant calculen. *Bottom-up* és simple de raonar, té accés $O(1)$ a qualsevol estat després de construir-se i el seu recompte d'estats és totalment predictable. A canvi, fa feina sobre estats que potser mai seran necessaris per a la configuració inicial.

Top-down és més selectiu: només calcula allò que la recursió necessita. És l'opció natural per a la GUI, perquè l'usuari normalment demana una sola configuració inicial i no consultes arbitràries sobre tota la taula. El cost és que cada estat memoitzat té més sobrecàrrega d'objectes, i que el temps depèn més de la forma concreta de l'espai accessible.

Per tant, la selecció final no és absoluta. Si es volen moltes consultes sobre la mateixa configuració global, *bottom-up* pot compensar el cost inicial. Si es vol resoldre una partida concreta i mostrar-ne una reproducció, la versió *top-down* sol fer menys feina.

6. Conclusions

La pràctica ha implementat una aplicació completa per resoldre el joc de la calculadora amb programació dinàmica. El model del problema queda capturat per l'estat (t, l, j) i per una recurrència que retorna el jugador perdedor sota joc òptim. La propietat que el total sempre augmenta permet evitar cicles i justifica tant la memoització *top-down* com la taula *bottom-up*.

L'arquitectura MVC ha separat la lògica del joc de la interfície Swing. Això ha facilitat validar el controlador sense obrir finestres i reutilitzar el mateix model per al benchmark. La GUI permet configurar la partida, alternar entre algorismes, usar teclats estàndard o aleatoris i seguir una línia òptima pas a pas.

Els resultats experimentals mostren que *bottom-up* calcula sempre $L \cdot (K + 1) \cdot P$ estats, mentre que *top-down* pot calcular-ne una fracció menor. Els temps de CPU creixen en general amb la mida de la instància, però no segueixen de manera directa el nombre d'estats: les constants d'implementació poden fer que *bottom-up* sigui més ràpid en alguns límits grans. En memòria, *top-down* redueix el nombre d'estats guardats, però el `HashMap` introdueix més sobrecàrrega per estat que els vectors primitius.

Com a treball futur es podrien afegir teclats editables manualment, comptar el nombre total de línies de joc possibles independentment de la línia òptima i ampliar el benchmark amb límits més grans o més jugadors per observar millor el factor P de la complexitat.

Referències

- [1] GeeksforGeeks, "Dynamic Programming or DP," 2026. [Online]. Available: <https://www.geeksforgeeks.org/dynamic-programming/>
- [2] Wikipedia contributors, "Model–view–controller," Wikipedia. [Online]. Available: <https://en.wikipedia.org/wiki/Model-view-controller>
- [3] GeeksforGeeks, "Introduction to Java Swing," 2025. [Online]. Available: <https://www.geeksforgeeks.org/introduction-to-java-swing/>